

C PROGRAMMING ASSIGNMENT

Scientific Calculator

Using Switch Statement

Documentation Report

Submitted by	Vishal Prajapati
Roll Number	FDAI044-A
Subject	Programming with C
Faculty In-charge	Dr. Kishor Mahajan
Class	FY-BSC AI
Date	25 August 2026

Contents

1. Assignment Overview
2. Problem Statement and Objectives
3. Input and Output Specification
4. Algorithm
5. Program Logic and Flowchart
6. Implementation Details
7. Source Code
8. Test Cases and Verified Output
9. Conclusion
10. References

1. Assignment Overview

This project implements a menu-driven scientific calculator using a switch statement. It performs addition, subtraction, multiplication, division, modulus, power, and percentage calculations. The program is designed as a console application so that the relationship between a numbered menu and fixed operation choices remains visible and easy to test.

Item	Description
Assignment	Assignment 1 — Scientific Calculator
Application type	Menu-driven console application
Primary C concepts	switch statement, arithmetic operators, double and int data types, pow(), input validation, do-while repetition
Execution model	Interactive input, validation, operation, and repeated menu processing
Compiler command	gcc source_file.c -lm

2. Problem Statement and Objectives

- Use a switch statement to select one operation from a numbered menu.
- Perform arithmetic operations with suitable numeric data types.
- Prevent division by zero and modulus by zero.
- Reject choices outside the valid menu range and clear invalid menu input.
- Allow multiple calculations until the user selects Exit.

3. Input and Output Specification

Menu option	Input required	Expected output
1 — Addition	Two real numbers	Their sum formatted to two decimals
2 — Subtraction	Two real numbers	First number minus second number
3 — Multiplication	Two real numbers	Their product formatted to two decimals
4 — Division	Dividend and divisor	Quotient, unless divisor is zero
5 — Modulus	Two integers	Integer remainder, unless divisor is zero
6 — Power	Base and exponent	Base raised to exponent
7 — Percentage	Value and percentage	Requested percentage of the value
8 — Exit	No additional input	Termination message

4. Algorithm

1. Display the calculator menu.

2. Read the user's menu choice. If the choice is not an integer, clear the invalid input and display an error message.
3. Use switch(choice) to route control to the selected operation.
4. For division and modulus, validate the second operand before evaluating the operator.
5. For modulus, read integer operands because the C modulus operator is used with integers.
6. Display the result or the relevant error message.
7. Repeat the menu until the user selects option 8, Exit.

5. Program Logic and Flowchart

The program begins with a do-while loop that displays the menu and reads a choice. The switch statement maps each fixed integer choice to one operation. The division and modulus branches perform zero-divisor checks before calculation. After the result or error message is displayed, control returns to the menu. Selecting option 8 displays the exit message and terminates the loop.

Flowchart symbol key

Rounded rectangle = Start/End; parallelogram = Input/Output; rectangle = Processing; diamond = Decision; circle = Connector.

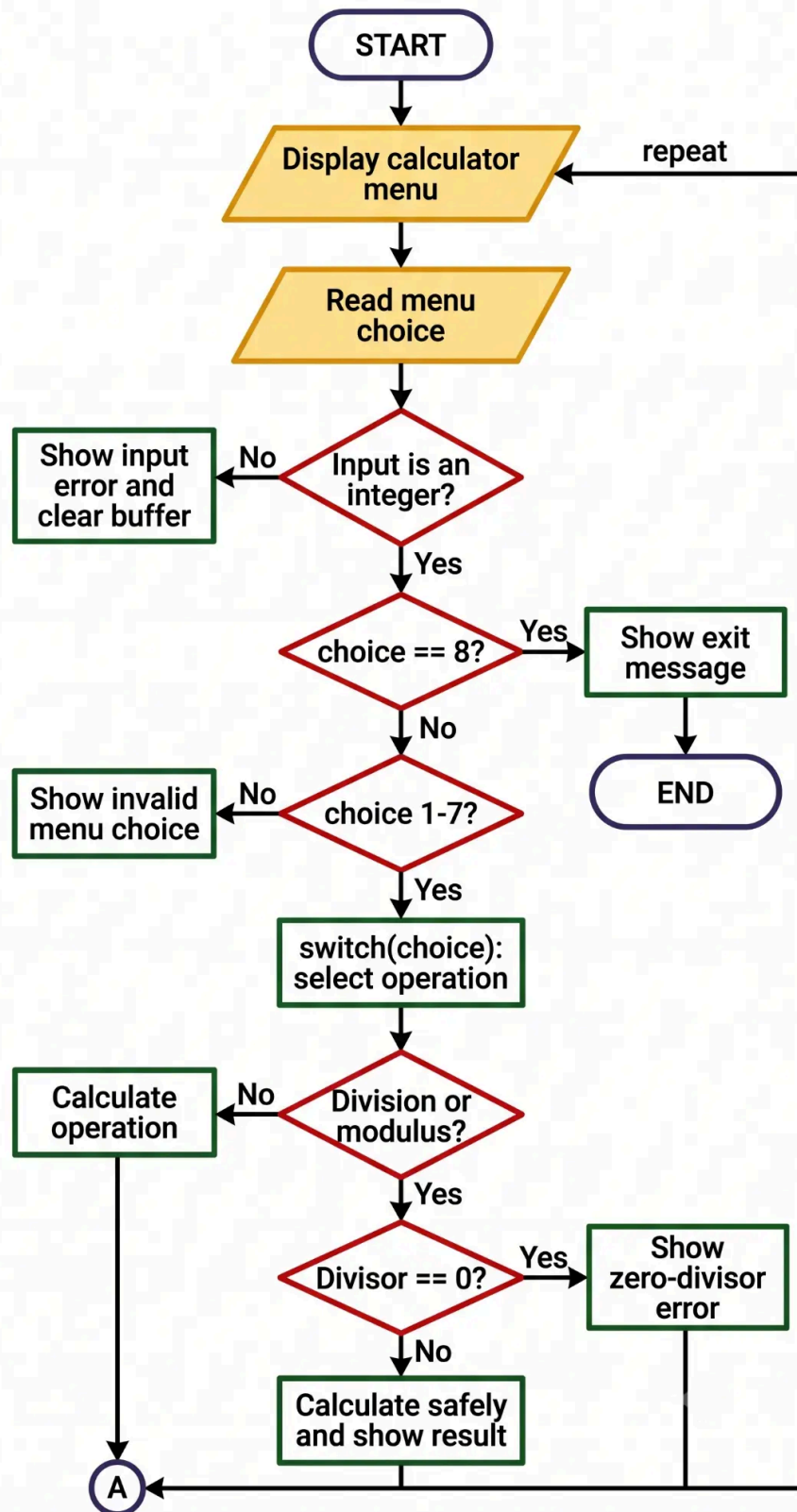


Figure 1. Scientific calculator flowchart using standard geometric symbols and directed arrows.

6. Implementation Details

The variables `a`, `b`, and `result` use the `double` type so that division, power, and percentage calculations can produce fractional results. The modulus operation uses `intA` and `intB` because the `%` operator is intended for integer operands in this program. The `pow()` function from `math.h` performs power calculation; therefore, the program is compiled with the math library using `gcc scientific_calculator.c -lm`.

Why switch is suitable here

The program compares one menu variable with several fixed integer constants. Each case corresponds directly to a numbered menu operation, making the structure easier to read and maintain than a long if-else-if chain. If the conditions involved ranges or complex logical expressions, if-else would be more suitable.

7. Source Code

```

1  #include <stdio.h>
2  #include <math.h>
3
4  int main(void) {
5      int choice;
6      double a, b, result;
7      int intA, intB;
8
9      do {
10         printf("\n===== Scientific Calculator =====\n");
11         printf("1. Addition\n");
12         printf("2. Subtraction\n");
13         printf("3. Multiplication\n");
14         printf("4. Division\n");
15         printf("5. Modulus\n");
16         printf("6. Power\n");
17         printf("7. Percentage\n");
18         printf("8. Exit\n");
19         printf("Enter your choice: ");
20
21         if (scanf("%d", &choice) != 1) {
22             printf("Invalid input. Please enter a number from 1 to 8.\n");
23             while (getchar() != '\n') {
24                 /* Clear invalid input from the keyboard buffer. */
25             }
26             continue;
27         }
28
29         switch (choice) {
30             case 1:
31                 printf("Enter two numbers: ");
32                 scanf("%lf %lf", &a, &b);
33                 printf("Result = %.2f\n", a + b);
34                 break;
35
36             case 2:
37                 printf("Enter two numbers: ");
38                 scanf("%lf %lf", &a, &b);
39                 printf("Result = %.2f\n", a - b);
40                 break;
41
42             case 3:
43                 printf("Enter two numbers: ");
44                 scanf("%lf %lf", &a, &b);
45                 printf("Result = %.2f\n", a * b);
46                 break;
47
48             case 4:
49                 printf("Enter dividend and divisor: ");
50                 scanf("%lf %lf", &a, &b);
51                 if (b == 0) {

```

```

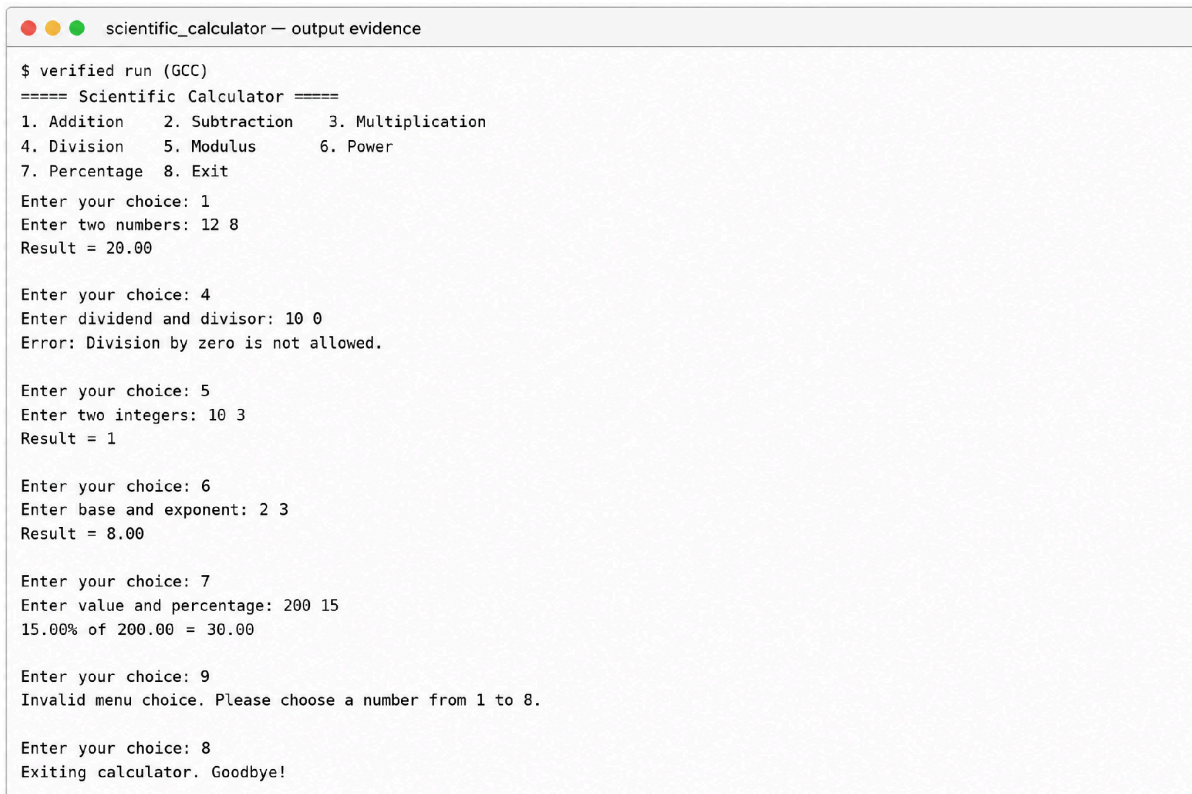
52     printf("Error: Division by zero is not allowed.\n");
53 } else {
54     printf("Result = %.2f\n", a / b);
55 }
56 break;
57
58 case 5:
59     printf("Enter two integers: ");
60     if (scanf("%d %d", &intA, &intB) != 2) {
61         printf("Invalid input. Modulus requires two integers.\n");
62         while (getchar() != '\n') {
63             /* Clear invalid input from the keyboard buffer. */
64         }
65     } else if (intB == 0) {
66         printf("Error: Modulus by zero is not allowed.\n");
67     } else {
68         printf("Result = %d\n", intA % intB);
69     }
70     break;
71
72 case 6:
73     printf("Enter base and exponent: ");
74     scanf("%lf %lf", &a, &b);
75     result = pow(a, b);
76     printf("Result = %.2f\n", result);
77     break;
78
79 case 7:
80     printf("Enter value and percentage: ");
81     scanf("%lf %lf", &a, &b);
82     result = (a * b) / 100.0;
83     printf("%.2f%% of %.2f = %.2f\n", b, a, result);
84     break;
85
86 case 8:
87     printf("Exiting calculator. Goodbye!\n");
88     break;
89
90 default:
91     printf("Invalid menu choice. Please choose a number from 1 to 8.\n");
92 }
93 } while (choice != 8);
94
95 return 0;
96 }

```

8. Test Cases and Verified Output

The following tests were executed after compilation with GCC. They cover normal operations, safety validation, invalid menu handling, and controlled termination.

Test	Input sequence	Observed / expected output	Purpose
1	Choice 1; 12 and 8	Result = 20.00	Addition
2	Choice 4; 10 and 0	Division-by-zero error	Safety validation
3	Choice 5; 10 and 3	Result = 1	Integer modulus
4	Choice 6; 2 and 3	Result = 8.00	Power calculation
5	Choice 7; 200 and 15	15.00% of 200.00 = 30.00	Percentage calculation
6	Choice 9	Invalid menu choice	Default case



```
$ verified run (GCC)
===== Scientific Calculator =====
1. Addition    2. Subtraction    3. Multiplication
4. Division    5. Modulus        6. Power
7. Percentage  8. Exit

Enter your choice: 1
Enter two numbers: 12 8
Result = 20.00

Enter your choice: 4
Enter dividend and divisor: 10 0
Error: Division by zero is not allowed.

Enter your choice: 5
Enter two integers: 10 3
Result = 1

Enter your choice: 6
Enter base and exponent: 2 3
Result = 8.00

Enter your choice: 7
Enter value and percentage: 200 15
15.00% of 200.00 = 30.00

Enter your choice: 9
Invalid menu choice. Please choose a number from 1 to 8.

Enter your choice: 8
Exiting calculator. Goodbye!
```

Figure 2. Verified console output showing calculator operations and validation responses.

9. Conclusion

The scientific calculator satisfies the required operations and uses switch as its primary decision structure. It includes checks for invalid menu choices, division by zero, and modulus by zero. The loop supports repeated calculations, while the verified console run confirms the documented outputs.

10. References

- [1] ISO C standard library reference, cppreference.com: <https://en.cppreference.com/w/c>
- [2] C switch statement reference, cppreference.com: <https://en.cppreference.com/w/c/language/switch>
- [3] C do-while loop reference, cppreference.com: <https://en.cppreference.com/w/c/language/do>
- [4] C mathematical functions reference, cppreference.com: <https://en.cppreference.com/w/c/numeric/math>